

01_项目代码仓库与工程结构说明

一、代码仓库基本情况

本项目代码仓库采用前后端分离结构，围绕“案件材料接入—证据图谱构建—图谱支撑检索—智能分析—侦查画像—证据溯源”的业务链路组织。后端以 FastAPI 为核心，提供案件管理、材料导入、证据地图、智能分析、画像报告、法律知识库和检察分析技能包等接口；前端基于 Vue3 + Vite + TypeScript，实现案件导入、图谱展示、智能分析、画像生成和溯源查看等交互页面。系统同时包含脱敏演示数据、法律知识库、结构化样本数据、截图脚本和运行部署配置，能够支撑比赛演示和功能验证。

二、工程结构说明

law_project/	
├─ backend/	后端服务
│ ├─ app/	
│ │ ├─ api/v1/routes/	FastAPI 路由层
│ │ ├─ services/	核心业务服务层
│ │ ├─ adapters/	算法与 HippoRAG 适配层
│ │ ├─ schemas/	Pydantic 数据契约
│ │ ├─ hipporag/	本地 HippoRAG 相关代码
│ │ └─ storage/	SQLite 与状态存储
│ └─ tests/	后端测试
│ └─ requirements.txt	Python 依赖
│ └─ README.md	后端运行说明
├─ frontend/	前端工程
│ ├─ src/	
│ │ ├─ views/	页面视图
│ │ ├─ components/	图谱、溯源、布局等组件
│ │ ├─ api/backend.ts	前端 API 封装
│ │ └─ router.ts	前端路由
│ └─ scripts/	Playwright 截图脚本
│ └─ package.json	前端依赖与启动脚本
├─ data/	数据目录
│ ├─ realtest/	原始演示案件
│ ├─ realcleantest/	脱敏演示案件
│ └─ legal_knowledge/	法律知识库
│ └─ structured/	结构化流水、通话、时间线数据
├─ docs/	项目文档
├─ outputs/	SQLite、截图、HippoRAG 输出
├─ scripts/	辅助脚本
└─ Caddyfile.jcmx	部署相关配置

三、后端代码说明

后端主要位于 backend/app 下。api/v1/routes 负责提供 HTTP 接口，覆盖案件管理、材料导入、图谱查询、证据地图、智能分析、画像报告、法律知识和技能包等功能；services 是业务服务层，承担材料接入、分析、图谱、画像、规则包、法律知识等核心逻辑；adapters 负责算法与 HippoRAG 适配；schemas 定义前后端数据契约；storage 负责 SQLite JSON payload 持久化。

其中，支撑方案蓝图和核心功能的重点文件包括：

```
backend/app/services/evidence_map_service.py
backend/app/services/analysis_service.py
backend/app/services/retrieval_planner_service.py
backend/app/services/report_service.py
backend/app/services/ingestion_service.py
backend/app/services/legal_knowledge_service.py
backend/app/services/analysis_skill_service.py
backend/app/adapters/algorithm.py
backend/app/storage/memory_store.py
```

四、前端代码说明

前端主要位于 `frontend/src` 下。`views` 目录承载核心页面，包括案件导入与管理、案件证据地图、智能分析、侦查画像、法律知识库等；`components` 目录包含图谱组件、G6 图谱渲染组件和页面通用组件；`api/backend.ts` 负责封装后端接口调用；`router.ts` 负责页面路由。图谱展示主要由 `Graph.vue` 和 `G6EvidenceGraph.vue` 实现，智能分析页面由 `Intelligence.vue` 实现，画像页面由 `Portrait.vue` 实现。

五、数据库与状态存储说明

当前系统没有使用传统 ORM 模型，而是采用 SQLite 表保存 JSON payload，同时保留内存字典缓存。SQLite 用于保存案件、证据、原文、抽取结果、图谱、画像、分析会话、检索过程、文件母图和规则运行记录，是原型阶段的统一状态仓库。相关数据模型包括 `cases`、`evidence`、`raw_contents`、`extractions`、`graphs`、`reports`、`portrait_facts`、`analysis_threads`、`analysis_messages`、`retrieval_sessions`、`retrieval_steps`、`document_mother_nodes`、`analysis_skill_registry`、`rule_pack_runs` 等。

六、代码与赛题要求的对应关系

本项目代码结构与赛题要求具有直接对应关系：

- 材料导入与处理 → `ingestion_service.py` / `information_extraction.py`
- 案件证据图谱 → `evidence_map_service.py` / `algorithm.py` / `Graph.vue`
- 图谱支撑机制 → `Graph.vue` / `G6EvidenceGraph.vue` / `retrieval_planner_service.py`
- 智能分析 → `analysis_service.py` / `Intelligence.vue`
- 侦查画像 → `portrait_facts` / `report_service.py` / `Portrait.vue`
- 证据溯源 → `GroundedSentence` / `PortraitFact` / `evidence_detail API`
- 法律实务知识 → `legal_knowledge_service.py` / `data/legal_knowledge`
- 检察 Skills → `analysis_skill_service.py` / `rule-packs API`

02_系统功能实现清单

功能模块	对应赛题要求	实现内容	代码 / 页面依据	实现状态
案件管理	办案场景承载	支持案件创建、案件列表、案件更新、案件删除	<code>cases.py</code> / <code>Dashboard</code>	已实现
多源材料导入	数据处理	支持文本、文件、多文件、ZIP 压缩包导入	<code>ingestion.py</code> / <code>ingestion_service.py</code>	已实现
文件解析	数据处理	支持 <code>.txt</code> 、 <code>.md</code> 、 <code>.csv</code> 、 <code>.xlsx</code> 、 <code>.json</code> 、 <code>.pdf</code> 、 <code>.docx</code> 等格式， <code>.doc</code> 暂不支持	<code>information_extraction.py</code>	已实现
证据切片与编号	数据处理	将材料解析为证据片段，并生成 <code>evidence_id</code>	<code>information_extraction.py</code> / <code>schemas/common.py</code>	已实现
实体与事实关系抽取	分析基础	抽取人员、单位、时间、地点、账户、金额、行为等要素及事实关系	<code>algorithm.py</code> / <code>HippoRAG</code> / <code>information_extraction.py</code>	部分实现
三层证据图谱	数据处理、分析	构建文件层、片段层、事实关系层证据图谱	<code>evidence_map_service.py</code> / <code>Graph.vue</code>	已实现
图谱可视化	人物关系、线索分析	支持 G6 图谱展示、图层切换、节点详情	<code>G6EvidenceGraph.vue</code> / <code>Graph.vue</code>	已实现
图谱筛选	分析、碰撞	支持按节点类型、关系类型筛选	<code>Graph.vue</code>	原型实现
核心邻域	分析、碰撞	围绕选中节点展示一跳关系	<code>buildCoreNeighborhoodSubgraph</code>	演示性实现
邻域扩展	分析、碰撞	基于前端 BFS 展示二跳关系	<code>buildExpandedNeighborhoodSubgraph</code>	演示性实现
多跳召回	线索分析	展示目标节点之间 2-4 跳路径	<code>buildMultihopPathSubgraph</code> / <code>shortestPath</code>	演示性实现
子图交叉	碰撞分析	展示两个对象或主题子图的交集与桥接边	<code>buildIntersectionSubgraph</code>	演示性实现
智能分析会话	分析、参考建议	支持假设验证、可疑资金流、多轮会话、继续追问、删除会话	<code>analysis_service.py</code> / <code>Intelligence.vue</code>	已实现

功能模块	对应赛题要求	实现内容	代码 / 页面依据	实现状态
Agentic RAG	分析、 线索分析	支持检索计划、工具调用、文档检索、图谱检索、 法律检索、节点扩展等	retrieval_planner_service.py	部分实现
检索过程保存	可解释分析	保存 RetrievalSession、RetrievalStep、ToolCall 等过程信息	memory_store.py / retrieval_planner_service.py	已实现
法律知识库	法律实务支撑	支持法律知识检索、罪名模板、办案流程知识	legal_knowledge_service.py	部分实现
检察分析 Skills	任务化分析	支持规则包列表与规则包运行，供 Agentic RAG 调用	analysis_skill_service.py / analysis_skills.py	部分实现
侦查画像	信息画像	生成关系叙述、行为叙述、关系事实、行为事实	portrait_facts / Portrait.vue	已实现
画像报告	参考建议	生成画像报告、建议和支撑事实	report_service.py / reports.py	已实现
证据溯源	可复核	智能分析句子、画像事实、报告 claim 可绑定证据片段或图谱路径	GroundedSentence / PortraitFact	已实现
人工编辑	人机协同	支持图谱节点和边的新增、修改、删除、复核	PATCH /graph/interventions	已实现
跨案碰撞	扩展能力	存在跨案合并图接口和 cross_case 疑点分析入口	/graph/merged / suspicion_analysis	原型实现
GNN 异常识别	未来增强	未见真实 GNN 代码	无	规划增强

当前功能状态中，材料导入、证据切片、三层证据图谱、智能分析、画像生成、证据溯源和人工编辑均可演示；邻域扩展、多跳召回、子图交叉主要属于前端基于已加载图谱的演示性图操作，后续可下沉到后端图算法；Agentic RAG 和检察分析 Skills 已接入主流程，技能包选择器、后端图数据库级碰撞和 GNN 异常识别仍可继续增强。

03_核心技术模块说明

一、证据接入与预处理模块

模块目标

该模块用于解决案件材料来源多、格式复杂、文本长、结构化记录数量大等问题。系统通过统一导入、解析、切片和编号，将原始卷宗、笔录、流水、通信等材料转化为后续可检索、可建图、可溯源的证据单元。

实现方式

系统提供文本导入、单文件导入、多文件导入和 ZIP 压缩包导入能力。材料进入系统后，先进行文件类型判断和内容解析，再对非结构化文本进行切句或切片，对结构化数据进行行级解析，并生成 evidence_id 保存入库。支持格式包括 .txt、.md、.csv、.xlsx、.json、.pdf、.docx，其中 .doc 暂不支持。

输出结果

模块输出包括证据文件元数据、原文内容、证据片段、结构化抽取结果和基础事实关系，为证据图谱构建和检索分析提供底座。

二、案件证据图谱构建模块

模块目标

该模块用于解决材料分散、事实要素隐藏、证据之间缺乏结构化关联的问题，将案件材料组织为“文件—片段—事实—关系”的案件证据图谱体系。

实现方式

系统通过 EvidenceMapService 和 EvidenceMapBuilder 构建证据地图，返回 documents、passages、triples、document_edges、passage_edges、containment_edges 等结构。文件层基于 EvidenceRecord 与 DocumentMotherNode 构建，片段层基于 ExtractionResult.passages 和原文切片构建，事实关系层基于 ExtractionResult.triples、HippoRAG OpenIE 合并图和本地抽取结果构建。三层图谱分别对应材料边界、证据定位和事实关系分析。

输出结果

形成文件层图谱、片段层图谱和事实关系层图谱。文件层回答“材料从哪里来”，片段层回答“证据在哪里”，事实关系层回答“事实如何关联”。这些图谱为后续关系扩展、智能分析和侦查画像提供结构化基础。

三、图谱支撑检索模块

模块目标

该模块用于解决普通检索只能找到相似文本、难以发现多跳关系和交叉线索的问题。系统通过图谱筛选、邻域扩展、多跳召回和子图交叉，把分散证据组织为可观察的局部证据网络。

实现方式

前端图谱页面由 `Graph.vue` 和 `G6EvidenceGraph.vue` 实现，系统支持文件层、片段层、事实关系层切换。图谱页中的核心邻域、邻域扩展、多跳召回和子图交叉，当前主要由前端基于已加载图谱数据进行过滤、BFS、路径搜索和交集计算实现。后端同时提供 `/graph`、`/evidence-map` 和 `/graph/interventions` 等接口，支持图谱数据获取和人工编辑。

输出结果

系统能够围绕目标对象、关键事件或侦查假设展示一跳关系、二跳扩展、多跳路径和交叉子图，辅助办案人员观察共同节点、关系路径、时间交叉和线索碰撞。

边界说明

当前图谱页的邻域扩展、多跳召回、子图交叉主要属于前端演示性图操作，不是完整后端图数据库查询；PPR 检索在 Agentic RAG / trace 中存在，但图谱页按钮本身未确认直接调用后端 PPR。

四、Agentic RAG 与任务化分析模块

模块目标

该模块用于解决大模型直接分析案件时证据范围不清、输出结构不稳定、结论难以核查的问题。系统通过检索计划、工具调用、证据组织和分析上下文构建，使大模型在任务约束和证据约束下生成分析结果。

实现方式

系统中的 Agentic RAG 检索计划器为 `RetrievalPlannerService`。它支持文档检索、图谱检索、节点扩展、时间范围检索、法律知识检索、规则包列表和规则包运行等工具。若模型配置可用，系统可调用 `planner` 模型生成 JSON 工具计划；若模型不可用或解析失败，则走 `fallback` 固定工具序列。检索过程被持久化为 `RetrievalSession` 和 `RetrievalStep`，并可在智能分析页面查看。

输出结果

系统可围绕用户提出的对象、行为、资金流、假设等问题，组织案件证据、图谱关系和法律知识，形成任务化分析上下文，再生成智能分析结果、行为还原、可疑信息和核查建议。

边界说明

系统对话入口采用 `@...` 作为任务上下文指定方式，可用于指定证据材料、法律知识和检索分析技能。检索 Skills 在后端以 `list_rule_packs`、`run_rule_pack` 等工具形式接入 Agentic RAG，前端可通过 `@行为画像分析` 等任务约束表达分析方向，并与规则包能力联动。

五、侦查画像生成模块

模块目标

该模块用于将案件证据、图谱关系、检索结果和法律知识整合为面向目标对象的侦查画像结果，支撑人物关系还原、行为方式分析和侦查参考建议生成。

实现方式

画像页面 `Portrait.vue` 调用 `POST /cases/{case_id}/analysis/portrait-facts`。后端 `AnalysisService.portrait_facts` 会调用 `RetrievalPlannerService.run(mode="portrait")`，收集 retrieval artifacts，再通过画像生成逻辑形成 `PortraitFactsResult`，前端展示人物关系、行为还原和证据溯源。画像输入包括案件标题、罪名、证据原文、抽取结果、图谱、Agentic RAG artifacts、法律知识 passages、图谱事实和文档检索 passage 等。

输出结果

画像结果包括 `relationship_narrative`、`behavior_narrative`、`relationship_facts`、`behavior_facts`、`tool_calls`、`retrieval_session_id` 等字段，对应人物关系还原、行为事实分析、可疑信息提示和画像建议。

六、证据溯源与人机协同模块

模块目标

该模块用于避免黑箱式智能输出，保证分析结论、画像事实和图谱关系能够回查原始材料和证据片段，并支持办案人员继续追问、人工修正和复核。

实现方式

智能分析句子使用 `GroundedSentence.supporting_passages/supporting_graph_paths` 保存来源，画像事实使用 `PortraitFact.passages/evidence_ids` 保存来源，画像报告 claim 使用 `PortraitClaim.supporting_passages` 保存支撑证据。图谱节点和边也可通过 `evidence_ids` 与证据原文关联，前端在 `Intelligence.vue`、`Portrait.vue` 和 `Graph.vue` 中提供溯源抽屉或详情面板。

输出结果

用户可以点击分析句、画像事实或图谱节点查看支撑证据、来源文件、证据片段和关系路径。需要说明的是，当前系统尽量将句子与 `passage`、图谱路径或 `evidence_id` 绑定，但并非所有生成句子都保证强证据支撑；若模型输出无法与 `passage` 匹配，支撑强度会降低或无支撑。

04_系统运行与演示说明

一、运行环境

系统分为后端和前端两部分。后端基于 Python 3.10 与 FastAPI，前端基于 Vue3 + Vite + TypeScript。运行前需要安装后端 Python 依赖和前端 npm 依赖，并根据需要配置模型 API Key、HippoRAG、SQLite 存储路径和 embedding 模型路径。

二、后端启动

```
cd C:\Users\adm14\Desktop\law_project\backend
C:\Users\adm14\AppData\Local\Programs\Python\Python310\python.exe -m uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

三、前端启动

```
cd C:\Users\adm14\Desktop\law_project\frontend
npm install
npm run dev
```

前端默认端口为 5173。

四、环境变量说明

DEEPSEEK_API_KEY	DeepSeek / HippoRAG LLM 调用
DEEPSEEK_ANALYSIS_MODEL	分析生成模型，默认 <code>deepseek-reasoner</code>
DEEPSEEK_PLANNER_MODEL	planner 模型，默认 <code>deepseek-chat</code>
ALGORITHM_PROVIDER=hipporag	启用 HippoRAG
HIPPORAG_EMBEDDING_MODEL	BGE-M3 路径
STORAGE_DB_PATH	SQLite 状态库路径

首次启动时，`MemoryStore.__post_init__` 会执行 `_initialize_db` 创建 SQLite 表。

五、推荐演示流程

1. 打开系统首页，创建或选择案件；
 2. 导入 `data/realcleantest.zip` 脱敏演示案件；
 3. 点击运行分析，完成材料处理和图谱构建；
 4. 进入 `/graph`，依次展示文件层、片段层和事实关系层；
 5. 在事实关系层展示核心邻域、邻域扩展、多跳召回和子图交叉；
 6. 进入 `/intelligence`，展示任务化智能分析和检索过程；
 7. 进入 `/portrait`，展示人物关系、行为事实和证据溯源。

这一路线覆盖数据导入、证据图谱、图谱支撑机制、智能分析、侦查画像和证据溯源，是比赛演示中最完整的推荐路径。

六、常见注意事项

1. `.doc` 文件暂不支持，可优先使用 `.docx`、`.pdf`、`.txt`、`.xlsx`、`.csv` 等格式。
 2. 若模型 API Key 未配置，Agentic RAG planner 可能走 fallback 计划。
 3. 图谱页的邻域扩展、多跳召回、子图交叉当前主要是前端演示性图操作，不等同于完整后端图数据库查询。
 4. `@...` 可用于在对话中指定证据、法律知识和检察分析技能，便于把用户任务转化为有约束的分析流程。
 5. GNN 异常识别当前属于规划增强方向，演示中不应宣称已实现。

05_数据来源、合规与边界说明

一、数据来源说明

本项目演示数据主要来源于脱敏样本数据和模拟生成数据。仓库中包含 `data/realtest`、`data/realcleantest`、`data/structured`、`data/legal_knowledge` 等目录，其中 `realcleantest` 用于脱敏演示案件，`structured` 用于结构化流水、通话、时间线等样本，`legal_knowledge` 用于法律知识库和罪名模板支撑。

二、合规定位

系统定位为检察侦查辅助工具，仅用于依法立案侦查后，对依法调取的数据进行辅助处理、组织、分析和展示。系统不替代办案人员作出事实认定、法律判断或侦查结论，而是提供可回查、可复核、可修正的侦查参考信息。

三、证据处理原则

系统强调证据来源绑定和结果可回查。材料导入后，系统保存 `evidence_id`、原文内容、抽取结果、证据片段和图谱关系；在智能分析和画像生成中，系统尽量将结论句、画像事实和报告 `claim` 绑定到 `supporting passages`、`graph paths` 或 `evidence_ids`，从而降低黑箱式生成风险。

四、功能边界说明

当前系统已经实现多源材料导入、证据切片、三层证据图谱、智能分析、画像生成、证据溯源和人工编辑等功能；实体与事实关系抽取、Agentic RAG、检察分析 Skills、法律知识库和可疑资金流等已形成可演示链路；邻域扩展、多跳召回和子图交叉主要属于演示性前端图操作；跨案碰撞为原型实现；GNN 异常识别属于后续增强方向。

五、后续完善方向

后续可围绕以下方向继续完善：

- 1. 提升图谱质量：优化实体识别、事实关系抽取、时间标注和证据来源绑定；
- 2. 增强图谱分析：将邻域扩展、多跳召回、子图交叉等能力下沉到后端图算法；
- 3. 完善检察 Skills：增强技能选择、技能编辑、技能测试和技能回放；
- 4. 优化法律知识库：统一前后端接口，增强罪名模板和实务规则调用；
- 5. 强化溯源质量：提升生成句与证据片段的绑定准确性；
- 6. 拓展异常识别：探索资金异常、关系异常和高风险节点识别；
- 7. 推进实务验证：结合更多脱敏样本和检察官反馈优化办案辅助体验。

06_实现状态与边界说明

为了保证支撑材料真实可信，本项目对功能实现状态作如下说明：

已实现：
多源材料导入、证据切片、三层证据图谱、图谱页面、智能分析会话、画像生成、证据溯源、人工编辑。

部分实现：
实体/事实关系抽取、Agentic RAG、检察分析 Skills、法律知识库、可疑资金流分析。

演示性实现：
核心邻域、邻域扩展、多跳召回、子图交叉等图谱支撑视图，目前主要基于前端图谱数据进行过滤、路径搜索和交集计算。

原型实现：
跨案合并图与 cross_case 疑点分析入口。

规划增强：
GNN 异常识别、后端图数据库级路径分析、跨案件长期索引、技能测试回放、稳定证据矩阵。

07_关键代码与代表性实现摘录

本节摘录项目中最能体现系统实现能力的核心代码片段。为便于支撑材料阅读，代码仅保留关键逻辑，完整实现以仓库对应文件为准。

一、多源材料导入与统一证据编号

对应文件： backend/app/services/ingestion_service.py

```
def ingest_file(self, case_id: str, filename: str, content: bytes, source_type: str) -> IngestionResult:
    resolved_type = resolve_source_type(source_type, filename)
    text = extract_content(source_type=resolved_type, raw_bytes=content)
    return self._save_evidence(
        case_id=case_id,
        title=filename,
        source_type=resolved_type,
        content=text,
        raw_bytes=content,
        filename=filename,
    )

def ingest_archive(self, case_id: str, filename: str, content: bytes) -> BatchIngestionResult:
    with zipfile.ZipFile(io.BytesIO(content)) as archive:
        for info in archive.infolist():
            if info.is_dir() or self._should_skip(info.filename):
                continue
            results.append(self.ingest_file(case_id, info.filename, archive.read(info), "unknown"))
```

说明：该片段体现系统将单文件、多文件和 ZIP 压缩包统一进入证据接入流程。进入 `_save_evidence` 后，系统生成 `evidence_id`，保存原文、证据元数据和抽取结果，为后续图谱与溯源提供稳定主键。

二、文档解析、结构化行解析与证据切片

对应文件：`backend/app/services/information_extraction.py`

```
def route_extraction(source_type: str, content: str, raw_bytes: bytes | None = None,
                    filename: str | None = None, evidence_id: str | None = None) -> ExtractionResult:
    resolved = resolve_source_type(source_type, filename)
    if resolved in STRUCTURED_TYPES:
        rows = parse_structured_content(source_type=resolved, content=content, raw_bytes=raw_bytes)
        triples = map_rows_to_triples(rows, evidence_id=evidence_id, filename=filename)
        passages = build_passages_from_triples(triples)
        return ExtractionResult(triples=triples, passages=passages, metadata={"rows_count": len(rows)})

    triples, passages = extract_unstructured_triples(content, evidence_id=evidence_id)
    return ExtractionResult(triples=triples, passages=passages, metadata={"chars": len(content)})
```

说明：该片段体现系统对结构化材料和非结构化文本采取不同处理路径。流水、通话、时间线等结构化数据按行解析；笔录、文书等非结构化文本切分为 `passage`，并进入事实关系抽取链路。

三、案件证据地图三层构建

对应文件：`backend/app/services/evidence_map_service.py`

```
def build(self) -> EvidenceMap:
    documents = self._build_documents()
    passages, triples, containment_edges = self._build_passages_and_triples()
    documents = [
        doc.model_copy(update={
            "passage_count": len(self.passages_by_doc.get(doc.id, [])),
            "triple_count": len(self.triples_by_doc.get(doc.id, [])),
            "quality_status": doc.quality_status
            if self.passages_by_doc.get(doc.id) and self.triples_by_doc.get(doc.id)
            else "needs_review",
        })
        for doc in documents
    ]
    document_edges = self._limit_document_edges(self._build_document_edges(documents, passages, triples))
    passage_edges = self._build_passage_edges(passages, triples)
    return EvidenceMap(
        case_id=self.case_id,
        documents=documents,
        passages=passages,
        triples=triples,
        document_edges=document_edges,
        passage_edges=passage_edges,
        containment_edges=containment_edges,
    )
```

说明：该片段是“案件证据地图”的核心。系统把同一案件材料组织为文件层、片段层和事实关系层，并生成文件边、片段边和包含边，支持从材料边界到事实关系的逐级观察。

四、图谱邻域、路径与交叉视图

对应文件：`frontend/src/views/Graph.vue`


```
function buildDemoSubgraph(nodes: GraphNode[], edges: GraphEdge[]) {
  const seed = selectedGraphNodeId.value || pickDefaultSeed(nodes);
  if (!seed) return { nodes: [], edges: [] };
  if (demoMode.value === 'coreNeighborhood') {
    return buildCoreNeighborhoodSubgraph(nodes, edges, seed);
  }
  if (demoMode.value === 'expandedNeighborhood') {
    return buildExpandedNeighborhoodSubgraph(nodes, edges, seed);
  }
  if (demoMode.value === 'multiHopPaths') {
    return buildMultiHopPathSubgraph(nodes, edges, seed);
  }
  return buildIntersectionSubgraph(nodes, edges, seed);
}
```

说明：该片段体现图谱页面围绕目标对象提供多种观察视角：核心邻域用于观察一跳直接关系，邻域扩展用于观察二跳外围关系，多跳召回用于展示关系路径，子图交叉用于观察两个主题网络的重合与桥接。

五、Agentic RAG 检索计划与工具调用

对应文件： backend/app/services/retrieval_planner_service.py

```
for round_index in range(1, max(settings.agentic_rag_max_rounds, 1) + 1):
    plan = self._plan_next_round(
        case_title=case_title,
        mode=mode,
        question=question,
        graph=graph,
        evidence=evidence,
        previous_summary=previous_summary,
        unresolved_gaps=unresolved_gaps,
        forced_evidence_ids=forced_evidence_ids if round_index == 1 else [],
        initial_goals=initial_goals or [],
    )
    tool_specs = _sanitize_tool_specs(plan.get("tool_calls"), mode, question, forced_evidence_ids)
    for spec in tool_specs[: max(settings.agentic_rag_max_tool_calls_per_round, 1)]:
        call = self._execute_tool(case_id=case_id, spec=spec, evidence=evidence,
                                   raw_contents=raw_contents, extractions=extractions,
                                   graph=graph, offense_id=offense_id)
        tool_calls.append(call)
```

说明：该片段体现系统不是让模型直接裸答，而是先生成检索计划，再执行文档检索、图谱检索、节点扩展、时间检索、法律知识检索等工具调用，逐轮构建分析上下文。

六、检索过程持久化

对应文件： backend/app/services/retrieval_planner_service.py

```
step = RetrievalStep(
    step_id=new_id("rstep"),
    session_id=session.session_id,
    round_index=round_index,
    retrieval_goals=[str(item)[:120] for item in _safe_list(plan.get("retrieval_goals"))],
    tool_calls=tool_calls,
    coverage_summary=coverage_summary,
    unresolved_gaps=unresolved_gaps,
    ready_to_answer=ready,
    created_at=now_utc(),
)
self.store.retrieval_steps.setdefault(case_id, []).append(step)
session = session.model_copy(update={
    "retrieval_steps_count": len(steps),
    "tool_call_summary": _tool_call_summary(steps),
    "status": "completed" if ready else "running",
})
```

说明：该片段体现系统会保存每轮检索目标、工具调用、覆盖摘要和信息缺口。前端可以展示“检索与调用细节”，让分析过程可审计、可复核。

七、智能分析会话与继续追问

对应文件： backend/app/services/analysis_service.py

```
def create_thread(self, case_id: str, payload: AnalysisThreadCreate) -> AnalysisThreadDetail:
    thread = AnalysisThread(
        thread_id=new_id("ath"),
        case_id=case_id,
        mode=payload.mode,
        title=payload.title or _thread_title(payload.initial_question, payload.mode),
        created_at=now_utc(),
        updated_at=now_utc(),
        message_count=0,
    )
    self.store.analysis_threads.setdefault(case_id, []).append(thread)
    self.store.flush_case(case_id)
    return self.add_thread_message(
        case_id,
        thread.thread_id,
        AnalysisThreadMessageCreate(question=payload.initial_question),
    )
```

说明：该片段体现智能分析页采用会话式结构，支持新建分析、继续追问和删除会话。每个会话可关联检索过程、工具调用摘要、回答句子和证据来源。

八、句级证据溯源

对应文件： backend/app/services/analysis_service.py

```
def _ground_answer_sentences(answer: str, passages: list[TracePassage],
                             graph_context: list[str]) -> list[GroundedSentence]:
    claims: list[GroundedSentence] = []
    for sentence in _split_fact_sentences(answer):
        scored = sorted(
            ((_score_sentence_against_passage(sentence, item.text), item) for item in passages),
            key=lambda item: item[0],
            reverse=True,
        )
        supporting = [item for score, item in scored[:4] if score > 0]
        claims.append(GroundedSentence(
            text=sentence,
            supporting_passages=supporting,
            supporting_graph_paths=_rank_graph_paths(sentence, graph_context),
        ))
    return claims
```

说明：该片段体现分析结果并非纯文本展示，而是将回答拆成句子，并为每一句匹配证据片段和图谱路径。前端点击结论句即可查看来源材料。

九、画像事实生成与溯源

对应文件： backend/app/services/analysis_service.py

```
retrieval_detail = self.retrieval_planner.run(
    case_id=case_id,
    mode="portrait",
    question=f"{case.title} 事实画像 人物关系 行为事实 时间线 结果后果",
    case_title=case.title,
    evidence=evidence,
    raw_contents=raw_contents,
    extractions=extractions,
    offense_id=case.offense_id,
    graph=graph,
)
passages, graph_context, legal_passages = collect_session_artifacts(retrieval_detail)
relationship_facts = _facts_from_hipporag_passages(passages, "relationship")
behavior_facts = _facts_from_graph(graph, "behavior") + _facts_from_hipporag_passages(passages, "behavior")
```

说明：该片段体现画像页以检索结果、图谱关系和证据片段为基础，生成关系事实、行为事实和画像叙述，并保留事实对应的证据来源。

十、图谱人工编辑与复核

对应文件： backend/app/services/graph_service.py

```
def apply_intervention(self, case_id: str, payload: GraphInterventionRequest) -> InvestigationGraph:
    graph = self.get_graph(case_id)
    if payload.action == "upsert_node" and payload.node:
        graph.nodes = [node for node in graph.nodes if node.node_id != payload.node.node_id]
        graph.nodes.append(payload.node)
    elif payload.action == "upsert_edge" and payload.edge:
        graph.edges = [edge for edge in graph.edges if edge.edge_id != payload.edge.edge_id]
        graph.edges.append(payload.edge)
    elif payload.action == "delete_node":
        graph.nodes = [node for node in graph.nodes if node.node_id != payload.node_id]
        graph.edges = [edge for edge in graph.edges if edge.source != payload.node_id and edge.target != payload.node_id]
```

说明：该片段体现系统支持办案人员对图谱节点和关系进行人工增删改查、复核和修正，保证算法抽取结果可以被人工校正。

十一、法律知识检索

对应文件： backend/app/services/legal_knowledge_service.py

```
def retrieve(self, query: str, offense_id: str | None = None, top_k: int = 8) -> TraceResult:
    docs, doc_titles = self._build_legal_docs(offense_id)
    terms = _query_terms(query)
    scored = []
    for index, doc in enumerate(docs):
        score = _score_text(doc, terms)
        if score > 0:
            scored.append((score, index, doc))
    scored.sort(key=lambda item: item[0], reverse=True)
    passages = [
        TracePassage(rank=rank + 1, score=score, evidence_id=f"legal_{index}",
            title=doc_titles.get(str(index), "法律知识"), text=_shorten(doc, 600))
        for rank, (score, index, doc) in enumerate(scored[:top_k])
    ]
    return TraceResult(query=query, passages=passages, paths=[])
```

说明：该片段体现法律知识库可按罪名模板、流程知识和证据标准召回相关内容，为智能分析和画像报告提供法律实务上下文。

十二、检察分析 Skills 规则包运行

对应文件： backend/app/services/analysis_skill_service.py

```
def run_rule_pack(self, case_id: str, pack_name: str) -> RulePackRunResult:
    checks = _load_checks(self.packs_dir, pack_name)
    mother_nodes = self.store.document_mother_nodes.get(case_id, [])
    evidence = self.store.evidence.get(case_id, [])
    findings: list[RuleFinding] = []
    not_triggered: list[NotTriggeredRule] = []
    for rule in checks.get("checks", []):
        result = self._evaluate_rule(rule, mother_nodes, evidence)
        if result:
            findings.append(result)
        else:
            not_triggered.append(NotTriggeredRule(rule_id=rule.get("id", ""), reason="当前证据结构未触发该规则"))
    return RulePackRunResult(skill_name=pack_name, findings=findings,
                              not_triggered=not_triggered, data_gaps=_data_gaps(mother_nodes))
```

说明：该片段体现检察分析 Skills 不是普通提示词，而是可被后端执行的规则包。规则运行结果可进入智能分析和证据结构提示，帮助系统发现材料缺口和流程异常。

08_提交版功能边界表述

为保证比赛支撑材料真实、稳妥，本项目对若干增强能力采用如下表述：

能力	提交版表述
Agentic RAG	已实现检索计划、工具调用、证据组织和过程持久化，支持文档、图谱、时间和法律知识等多源召回。
检察 Skills	已实现规则包注册、列表、运行和对话任务约束接入，用于材料完整性、流程转向、权限控制和行为画像等分析方向；技能库管理可继续完善。
图谱路径与交叉	已提供核心邻域、邻域扩展、多跳路径和子图交叉等图谱观察视角，用于演示复杂关系梳理和线索碰撞。
证据溯源	分析句、画像事实和报告结论均尽量绑定证据片段、图谱路径或 evidence_id，支持回查原始材料。
可疑资金流	当前以结构化流水、通话记录、图谱关系和智能分析为主；更复杂的 GNN 异常识别可作为后续增强方向。

以上表述既能体现系统已经完成的技术路径，也避免把原型能力包装成已经完全工程化的生产系统。